

Honors Topics: Data Structures & Design

Comprehensive Review Packet & Reference Guide

1. Arrays

Array Elements:	[2]	[4]	[10]	[5]	[15]	[3]
Array Indexes:	0	1	2	3	4	5

1.1 Introduction to Arrays

An array is a fundamental data structure that stores a fixed-size sequential collection of elements of the same type. In Java, arrays are treated as objects.

- **Fixed size:** Static capacity determined upon instantiation; cannot be resized.
- **Homogeneous elements:** All elements must share the same data type.
- **Contiguous memory:** Elements are allocated in consecutive memory addresses.
- **Zero-indexed:** Valid array indices range from `0` to `length - 1`.

Declaration and Initialization:

```
// Declaration
int[] numbers;

// Instantiation (allocates 5 integers initialized to default 0)
int[] numbers = new int[5];

// Direct inline initialization
int[] numbers = {1, 2, 3, 4, 5};
```

1.2 Accessing and Modifying Array Elements

Array elements are accessed directly using square brackets `[]` with zero-based indexing in constant $O(1)$ time complexity.

```
int[] numbers = {10, 20, 30, 40, 50};
System.out.println(numbers[2]); // Outputs: 30

numbers[2] = 35; // Modifying an element
System.out.println(numbers[2]); // Outputs: 35
```

1.3 Array Operations & Time Complexities

Operation	Time Complexity
Access by index	$O(1)$
Insert / delete at beginning	$O(n)$
Insert / delete at end	$O(1)$
Insert / delete in middle	$O(n)$
Search in unsorted array	$O(n)$
Search in sorted array (Binary Search)	$O(\log n)$

1.4 Multi-Dimensional Arrays

Multi-dimensional arrays represent grids or matrices and are implemented in Java as arrays of arrays.

```
int[][] matrix = {
    {1, 2, 3},
    {4, 5, 6},
    {7, 8, 9}
};

int[][] matrix2 = new int[3][3];

System.out.println(matrix[1][2]); // Outputs: 6
System.out.println(matrix2[1][2]); // Outputs: 0 (default value)
```

1.5 Iterating Through an Array

```
int[] numbers = {1, 2, 3, 4, 5};

// Standard for loop
for (int i = 0; i < numbers.length; i++) {
    System.out.println(numbers[i]);
}

// Enhanced for loop (for-each)
for (int num : numbers) {
    System.out.println(num);
}
```

1.6 Limitations & Considerations

- **Fixed Capacity:** Cannot expand or shrink dynamically after creation.
- **Shift Overhead:** Inserting or deleting in the beginning/middle requires $O(n)$ element shifting.
- **No Built-in Helpers:** Lacks helper methods like `add()` or `remove()` found in `ArrayList`.

1.7 Common Pitfalls and Best Practices

- **ArrayIndexOutOfBoundsException** : Ensure all index values satisfy `0 ≤ index < length`.
- **NullPointerException** : Ensure object arrays and individual elements are instantiated prior to access.
- **Array vs. ArrayList**: Prefer arrays for fixed-size, performance-critical memory layouts; use `ArrayList` for dynamic sizing.

1.8 Advanced Topics

Array as Base for Other Structures: Simple arrays serve as the underlying storage mechanism for dynamic arrays (`ArrayList`), hash table bucket arrays (`HashMap`), and binary heaps.

Algorithm Analysis: Arrays provide contiguous memory locality, enabling predictable cache hits and straightforward $O(n)$ space analysis.

2. Linked Lists

```

HEAD ---> [ Data | Next ] ---> [ Data | Next ] ---> [ Data | Next ] ---> NULL
                Node           Node           Node
  
```

2.1 Introduction to Linked Lists

A linked list is a linear data structure where elements (nodes) are linked using pointers/references rather than stored in contiguous memory addresses.

- **Dynamic Sizing:** Allocates memory on demand without pre-allocated capacity limits.
- **Efficient Mutations:** Constant time $O(1)$ insertions and deletions at known pointer locations.
- **Sequential Access:** Requires $O(n)$ sequential traversal from the `head` node (no direct indexing).

2.2 Types of Linked Lists

- **Singly Linked List:** Each node contains data and a single `next` reference pointing to the subsequent node.
- **Doubly Linked List:** Each node contains data, a `next` reference, and a `prev` reference for bidirectional traversal.
- **Circular Linked List:** The final node points directly back to the `head` node.

2.3 Basic Operations & Time Complexities

Operation	Singly Linked List	Doubly Linked List
Insertion at beginning	$O(1)$	$O(1)$
Insertion at end	$O(n)$ *	$O(1)$ **
Deletion at beginning	$O(1)$	$O(1)$
Deletion at end	$O(n)$	$O(1)$
Search / Access by index	$O(n)$	$O(n)$

* $O(1)$ if maintaining a tail pointer. ** Assuming a tail pointer is maintained.

2.4 Implementation in Java

```
// Basic Node Structure
public class Node {
    int data;
    Node next;

    public Node(int data) {
        this.data = data;
        this.next = null;
    }
}

// Inserting a Node at the Beginning
public void insertAtBeginning(int data) {
    Node newNode = new Node(data);
    newNode.next = head;
    head = newNode;
}

// Traversing a Linked List
public void traverse() {
    Node current = head;
    while (current != null) {
        System.out.print(current.data + " -> ");
        current = current.next;
    }
    System.out.println("null");
}
```

2.5 Common Algorithms & Java Implementation

- **Runner Technique (Fast & Slow Pointers):** Uses two pointers moving at different speeds for cycle detection and finding list midpoints.
- **Reversing a List:** In-place pointer manipulation algorithm flipping links in $O(n)$ time.
- **Java's `java.util.LinkedList`:** Built-in doubly-linked list implementation of the `List` and `Deque` interfaces.

3. Stacks and Queues

3.1 Core Principles

- **Stack:** Last-In-First-Out (**LIFO**) principle. Core operations: `push`, `pop`, `peek`.
- **Queue:** First-In-First-Out (**FIFO**) principle. Core operations: `add`, `remove`, `peek`.

3.2 Time Complexities

Operation	Stack Complexity	Queue Complexity
Push / Add	$O(1)$	$O(1)$
Pop / Remove	$O(1)$	$O(1)$
Peek	$O(1)$	$O(1)$

3.3 Code Examples

```
// Stack Example: Balanced Brackets Checker
public static boolean isBalanced(String s) {
    Stack<Character> stack = new Stack<>();
    for (char c : s.toCharArray()) {
        if (c == '{') {
            stack.push(c);
        } else if (c == '}') {
            if (stack.isEmpty()) return false;
            stack.pop();
        }
    }
    return stack.isEmpty();
}

// Queue Example: Document Print Buffer
Queue<String> printQueue = new LinkedList<>();
printQueue.add("Document1");
printQueue.add("Document2");

while (!printQueue.isEmpty()) {
    System.out.println("Printing: " + printQueue.remove());
}
```

4. Hashes and Hash Maps

4.1 Hash Functions & Hash Maps

A hash function maps keys of arbitrary size to fixed integer hash values. Hash maps provide average-case **$O(1)$** time performance for key lookup, insertion, and deletion.

- **Deterministic:** Identical input keys always produce the exact same hash output.
- **Uniform Distribution:** Distributes keys evenly across hash buckets to minimize collisions.
- **Efficient:** Computes hash values in constant **$O(1)$** time.

4.2 Collision Resolution, Load Factor & Resizing

- **Collisions:** Occur when two distinct keys yield identical bucket indices. Resolved via *Chaining* (linked lists per bucket) or *Open Addressing* (probing).
- **Load Factor:** Ratio defined as `Load Factor = Stored Elements / Total Buckets`.
- **Resizing & Rehashing:** Java's `HashMap` defaults to an initial capacity of 16 buckets and a 0.75 load factor threshold. When exceeded, bucket capacity doubles and all elements are rehashed in amortized **$O(1)$** insertion time (**$O(n)$** during resize).

```
HashMap<String, Integer> map = new HashMap<>();
map.put("apple", 1);
map.put("banana", 2);

System.out.println(map.get("apple")); // Outputs: 1
map.remove("apple");
```

5. Graphs and Trees

5.1 Graphs & Traversal Algorithms

- **Directed Graphs:** Edges possess specific direction (e.g., Twitter follows).
- **Undirected Graphs:** Edges represent symmetric two-way connections (e.g., Facebook friends).
- **Depth-First Search (DFS):** Explores deeply along branches before backtracking. Implemented via a **Stack** or recursion.
- **Breadth-First Search (BFS):** Traverses neighbor nodes level-by-level. Implemented via a **Queue**; computes shortest paths in unweighted graphs.

5.2 Trees & Binary Search Trees (BST)

- **Tree Structure:** Connected acyclic graphs containing exactly $n - 1$ edges for n nodes with a unique simple path between any two nodes.
- **BST Property:** For every node, all left subtree keys are strictly smaller, and all right subtree keys are strictly larger.
- **Importance of Tree Balancing:** Balanced search trees guarantee $O(\log n)$ operations. Unbalanced search trees can degenerate into linear $O(n)$ structures.

6. Recursion, Greedy Algorithms, & Dynamic Programming

6.1 Recursion & Greedy Algorithms

- **Recursion:** Solves problems by having a method call itself on smaller instances. Must contain a *Base Case* to terminate and a *Recursive Case* making progress towards the base case.
- **Greedy Algorithms:** Makes locally optimal choices at each step aiming to reach a global optimum. Efficient, though not guaranteed to find optimal solutions for all problems.

6.2 Dynamic Programming (Memoization Example)

Dynamic Programming solves complex problems by breaking them into overlapping subproblems, computing each subproblem once, and caching results (memoization) to avoid redundant computation.

```
import java.util.HashMap;
import java.util.Map;

public class FibonacciDP {
    private static Map<Integer, Integer> memo = new HashMap<>();

    public static int fib(int n) {
        if (n <= 1) return n; // Base cases: fib(0) = 0, fib(1) = 1

        // Correct Memoization: Return immediately on cache hit
        if (memo.containsKey(n)) {
            return memo.get(n);
        }

        // Compute and store result
        int result = fib(n - 1) + fib(n - 2);
        memo.put(n, result);

        return result;
    }
}
```